

Distributed Job Scheduler

Complete Step-by-Step Implementation Guide

Java 17

Spring Boot 3

PostgreSQL

React 18

TypeScript

Docker

JUnit 5

Mockito

Tailwind CSS

Recharts

SYSTEM ARCHITECTURE

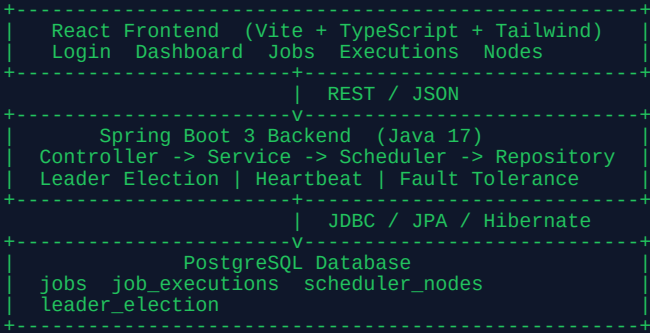


TABLE OF CONTENTS

1. Project Overview and Goals

2. Technology Stack - Backend Skills and Libraries

3. Technology Stack - Frontend Skills and Libraries

4. Development Tools Reference

5. Database Schema Design

6. Step-by-Step: Backend Implementation

7. Step-by-Step: Leader Election Algorithm

8. Step-

by-
Step:
Heartb
eat
and
Fault
Tolera
nce

9.

Step-
by-
Step:
Fronte
nd Im
pleme
ntation

10.

Step-
by-
Step:
Testin
g Strat
egy
(JUnit
5 + M
ockito)

11.

Step-
by-
Step:
Docke
r and
Contai
nerizat
ion

12.

REST
API R
eferen
ce

13.

Key D
esign
Decisi
ons
and P
attern
s

14.

R
unning
the A
pplicat
ion

1. PROJECT OVERVIEW AND GOALS

A Distributed Job Scheduler manages scheduled task execution across multiple server nodes. Nodes compete for cluster leadership via a database-backed distributed lock. Only the elected leader executes jobs, preventing duplicate runs. If the leader fails, another node takes over within seconds via the heartbeat/expiry mechanism.

Core Objectives

fault Tolerance - no single point

of failure; any node can become a leader

%, leader Election - database distribution

ed
lo
ck
us
in
g
S
EL
E
C
T
F
O
R
U
P
D
AT
E

%,

ea
rtb
ea
t-
ea
ch
n
od
e
pi
ng
s
ev
er
y
5s
;s
tal
e
no
de
s
au
to-
de
te
ct
ed

in
15
s
%,
ro
n
Sc
he

du
lin
g
-j
ob
s
on
c
us
to
mi
za
bl
e
cr
on
e
xp
re
ssi
on
s (
Sp
rin
g
6-f
iel
d f
or
m
at)
%,
ea
l-T
im
e
D
as
hb
oa
rd
-li
ve
a
d
mi
n
UI
: cl
us
ter
h
ea
lth
, e
xe
cu

tion
n
hi
st
or
Y,
ch
art
s
%,
ull
R
E
S
T
A
PI
-
C
R
U
D
wit
h
se
ssi
on
-b
as
ed
S
pri
ng
S
ec
uni
ty
au
th
en
tic
ati
on
%,
on
tai
ne
riz
ati
on
-
m
ult
i-s
ta
ge
D

oc
ke
r b
uil
ds
; d
oc
ke
r-c
o
m
po
se
fo
r
m
ult
i-n
od
e
si
m
ul
ati
on
%,
U
nit
5
+
M
oc
kit
o
Te
st
s
- u
nit
te
st
s f
or
se
rvi
ce
s,
M
oc
k
M
vc
te
st
s f
or
co

ntr
oll
er
s

What Makes It Unique

%,

ult
i-n
od
e
si
m
ul
ati
on
vi
a
do
ck
er-
co
m
po
se
: 2
+
sc
he
du
ler
n
od
es
c
o
m
pe
te
for
le
ad
er
sh
ip
lo
ca
lly

%,

pti
mi
sti
c
da
ta
ba
se

locking for reader selection - no Zookeeper, etcd, or Redis is required, **%,** version 5.5 shows real-time election cluster statistics each

an
ge
s
%,
oo
t s
eq
ue
nc
e
an
im
ati
on
o
n l
og
in
pa
ge
si
m
ul
at
es
s
ys
te
m
ini
tia
liz
ati
on
%,
pe
nA
PI
-fir
st
de
vel
op
m
en
t
ba
ck
en
d
co
ntr
ac
t a
ut
o-
ge

2. TECHNOLOGY STACK - BACKEND SKILLS

Java 17 (LTS)

ec
or
ds
fo
r i
m
m
ut
ab
le
da
ta
ca
rri
er
s;
se
al
ed
cl
as
se
s f
or
ex
ha
us
tiv
e
pa
tte
rn
m
at
ch
in
g
ex
t b
lo
ck
s f
or
m
ult
i-li
ne
st
rin
gs

in
te
st
s
an
d
JS
O
N
pa
yl
oa
ds
%,
nn
an
ce
d
sw
itc
h
ex
pr
es
si
on
s; i
m
pr
ov
ed
St
re
a
m/
C
oll
ec
tor
A
PI
s
%,
ar
for
lo
ca
lit
yp
e i
nf
er
en
ce
; i
ns
ta

nc
eo
f p
att
er
n
m
at
ch
in
g r
ed
uc
es
c
as
tin
g

Spring Boot 3.2

%,

ut
o-
co
nfi
gu
rat
io
n
eli
mi
na
te
s
bo
lle
rpl
at
e;
e
m
be
dd
ed
T
o
m
ca
tr
un
s
on
p
ort
8
08
0

%,

@

Sp

rin

gB

oo

tA

pp

lic

ati

on

tri

gg

er

s

co

m

po

ne

nt

sc

an

, J

PA

, s

ec

uri

ty,

ac

tu

at

or

%,

pin

ng

-b

oo

t-s

tar

ter

-v

ali

da

tio

n

pr

ovi

de

s

@

N

ot

Bl

an

k,

@

Si

ze
o
n r
eq
ue
st
bo
di
es
%,
pri
ng
A
ct
ua
tor
e
xp
os
es
/a
pi/
ac
tu
at
or/
he
alt
h f
or
D
oc
ke
r h
ea
lth
c
he
ck
s

Spring Data JPA and Hibernate

%,
ep
os
ito
ry
pa
tte
rn:
J
pa
R
ep
os
ito
ry

<Entity ID> provides full CRUD for free %, @Entity + @Table + @Column maps Java classes directly to PostgreSQL tables

%,

D
L
au
to-
up
da
te
cr
ea
te
s
an
d
alt
er
s t
ab
le
s
on
st
art
up
-
no
m
igr
ati
on
s
cri
pt
s
ne
ed
ed
%,
@
Lo
ck
(P
E
S
SI
MI
S
TI
C
-
W
RI
T
E)
is
su
es

S
EL
E
C
T
F
O
R
U
P
D
AT
Ef
or
at
o
mi
cl
ea
de
re
le
cti
on
%,
@
Q
ue
ry
wit
h
JP
Q
Lf
or
ag
gr
eg
ati
on
:c
ou
nt
by
st
at
us
,e
xe
cu
tio
ns
gr
ou
pe
d
by

d
ay
%,
@
Tr
an
sa
cti
on
al
en
su
re
s
at
o
mi
cit
y
of
le
ad
er
el
ec
tio
n r
ea
d-t
he
n-
wr
ite
o
pe
rat
io
ns

Spring Security

%,
es
si
on
-b
as
ed
a
ut
he
nti
ca
tio
n
-s
tat
ef

ul
H
TT
P
se
ssi
on
s
vi
a
Ht
tp
Se
ssi
on
%,
In-
m
e
m
or
y
Us
er
D
et
ail
sS
er
vic
e
wit
h
ad
mi
n
cr
ed
en
tia
ls
fro
m
en
vir
on
m
en
t v
ari
ab
le
s
%,
S
R
F

disabled for REST API; CORS headers configured for frontend origin
%,
© With MockUserver annotation in tests by passes real

au
th
for
c
on
tro
lle
r u
nit
te
st
s

Spring Scheduler (@Scheduled)

%,

@
En
ab
le
Sc
he
du
lin
g
ac
tiv
at
es
th
e
sc
he
du
lin
g
su
bs
ys
te
m
in
th
e
ap
pli
ca
tio
n
co
nt
ex
t

%,

@
Sc
he
du

le
d(f
ixe
d
D
el
ay
=5
00
0)
dri
ve
s t
he
h
ea
rtb
ea
t ti
m
es
ta
m
p
up
da
te
ev
er
y
5
se
co
nd
s
%,
@
Sc
he
du
le
d(f
ixe
d
D
el
ay
=1
00
00
) d
riv
es
le
ad
er
el

ec
tio
n
an
d j
ob
di
sp
at
ch
e
ve
ry
10
s
%,
ep
ar
at
e
Th
re
ad
Po
ol
Ta
sk
Ex
ec
ut
or
ru
ns
a
ct
ua
l j
ob
lo
gi
c
co
nc
urr
en
tly

Multithreading and Concurrency

%,
ol
ati
le
bo
ol
ea
n
cu

re
ntl
yL
ea
de
r e
ns
ur
es
cr
os
s-t
hr
ea
d
vis
ibil
ity
wi
th
ou
t s
yn
ch
ro
ni
za
tio
n
%,
hr
ea
dP
oo
IT
as
kE
xe
cu
tor
wi
th
co
nfi
gu
ra
bl
e
po
ol
siz
e f
or
co
nc
urr
en

t)
ob
e
xe
cu
tio
n
%,
es
si
mi
sti
c
D
B l
oc
k
at
tra
ns
ac
tio
n
bo
un
da
ry
pr
ev
en
ts
ra
ce
c
on
dit
io
ns

in
el
ec
tio
n
%,
@
Tr
an
sa
cti
on
al(
is
ol
ati
on
=S

3. TECHNOLOGY STACK - FRONTEND SKILLS

React 18 + TypeScript

F, unct ion c o m po ne nt s wit h ho ok s: us eS tat e, us eE ffe ct, u se C all ba ck, u se R ef % , yp eS cri pt str ict m od e: no i m pl ic it a ny ; fu ll

ty
pe
s
af
et
y
on
al
l A
PI
re
sp
on
se
s
%,
us
to
m
ho
ok
s f
or
au
th
st
at
e
m
an
ag
e
m
en
t a
nd
p
olli
ng
in
ter
val
c
on
fig
ur
ati
on
%,
ou
ter
: li
gh
tw
ei
gh
t 1

.5
K
B
cli
en
t-s
id
e r
ou
ter
, n
o
R
ea
ct
R
ou
ter
o
ve
rh
ea
d

TanStack Query v5 (React Query)

%,
se
Q
ue
ry
an
d
us
e
M
ut
ati
on
fo
r a
ll s
er
ve
r s
tat
e
m
an
ag
e
m
en
t
%,
fet
ch
int

er
val
: 5
00
0
au
to-
po
lis
da
sh
bo
ar
d,
no
de
s,
an
d
ex
ec
uti
on
s
ev
er
y
5
se
co
nd
s
%,
ac
he
in
val
id
ati
on
af
ter
m
ut
ati
on
s (
pa
us
e,
de
let
e,
tri
gg
er)
vi

a i
nv
ali
da
te
Q
ue
rie
s
%,
try
:f
al
se
o
n
au
th
ch
ec
k
pr
ev
en
ts
flo
od

of
40
1 r
etr
ie
s
on
in
iti
al
un
au
th
en
tic
at
ed
lo
ad

Orval + OpenAPI Code Generation

%,
pe
nA
PI
3.
0
sp
ec

in
lib
/a
pi-
sp
ec
/o
pe
na
pi-
ya
ml
d
efi
ne
s t
he
fu
ll
A
PI
co
ntr
ac
t
%,
np
m
-fi
lte
r
@
w
or
ks
pa
ce
/a
pi-
sp
ec
ru
n
co
de
ge
n r
eg
en
er
at
es
al
l h
oo
ks

a
ut
o
m
ati
ca
lly
%,
en
er
at
ed
R
ea
ct
Q
ue
ry
ho
ok
s
in
lib
/a
pi-
cli
en
t-r
ea
ct/
sr
c/
ge
ne
rat
ed
/-
ne
ve
r h
an
d-
ed
it
%,
ac
ke
nd
A
PI
ch
an
ge
s
br
ea
k t

he
fr
on
te
nd

at
Ty
pe
Sc
rip
t c
o
m
pil
e t
im
e,
no
t
at
ru
nti
m
e

Tailwind CSS v4

%,
tilit
y-f
irs
t
C
S
S
-n
o
cu
st
o
m
st
yl
es
he
et
s f
or
st
an
da
rd
lay
ou
t/s
pa
ci

ng
/c
ol
or
%,
S
S
cu
st
o
m
pr
op
ert
ie
s
(-
pri
m
ar
y
-c
ar
d,
-b
or
de
r)
po
w
er
th
e
da
rk
ter
mi
na
l t
he
m
e
%,
et
Br
ai
ns
M
on
o
an
d
Sp
ac
e
M
on

o
m
on
os
pa
ce
fo
nt
s t
hr
ou
gh
ou
t f
or
ha
ck
er
ae
st
he
tic
%,
us
to
m
ke
yfr
a
m
es
: s
ca
nli
ne
s
ov
erl
ay,
gr
id
ba
ck
gr
ou
nd
p
att
er
n,
cu
rs
or-
bli
nk
a
ni

in
ati
on

Recharts

%,

re
a
C
ha
rt f
or
ex
ec
uti
on
hi
st
or
y
ov
er
ti
m
e
wit
h
cy
an
gr
ad
ie
nt
fill
an
d
cu
st
o
m
to
olt
ip

%,

ar
C
ha
rt f
or
jo
bs
b
y
st
at
us
(

Ac
tiv
e/
Pa
us
ed
/Di
sa
bl
ed
)
wit
h
pe
r-b
ar
co
lor
c
od
in
g
%,
es
po
ns
ive
C
on
tai
ne
r d
yn
a
mi
ca
lly
fill
s
ca
rd
wi
dt
h
on
a
ny
vi
ew
po
rt
%,
ne
ar
Gr
ad
ie

nt
de
fs
in
si
de
c
ha
rt
S
V
G
for
gl
ow
in
g
ar
ea
ef
fe
ct

React Hook Form + Zod

%,
se
Fo
rm
wi
th
zo
d
R
es
ol
ve
r
wi
re
s
Zo
d
sc
he
m
a
val
id
ati
on
di
re
ctl
y
to
for
m

st
at
e
%,
ea
l-ti
m
e
pe
r-fi
el
d
err
or
m
es
sa
ge
s
wit
ho
ut
wr
iti
ng
m
an
ua
l v
all
da
tio
n
ha
nd
ler
s
%,
od
s
ch
e
m
as
e
nf
or
ce
ty
pe
s,
mi
n/
m
ax
le
ng

th
s,
an
d
cr
on
e
xp
re
ssi
on
p
att
er
ns

at
ru
nti
m

4. DEVELOPMENT TOOLS REFERENCE

Tool	Category	Purpose
Java 17 (GraalVM/OpenJDK)	Runtime	Java bytecode execution
Maven 3.9	Build	Dependency mgmt, fat JAR packaging, test runner
pnpm 9	Build	Fast monorepo package manager
Node.js 20+	Runtime	Frontend dev server, Orval codegen
Postman / curl	API Testing	Manual REST endpoint exploration
Docker + Compose	Container	Multi-node simulation, production packaging
JUnit 5	Testing	Java unit and integration test framework
Mockito	Testing	Mock dependencies for isolated unit tests
H2 Database	Testing	In-memory PostgreSQL-compatible DB for tests
SLF4J + Logback	Logging	@Slf4j structured log output
Spring Actuator	Monitoring	Health endpoint at /api/actuator/health
Orval	Codegen	OpenAPI to React Query hook generator

5. DATABASE SCHEMA DESIGN

migration scripts needed for d

Step 5 - Service Layer

7. STEP-BY-STEP: LEADER ELECTION ALGORITHM

Leader election uses a single row in the leader_election table as a distributed mutex. Any node that atomically acquires or renews that row becomes the leader. The key is SELECT FOR UPDATE which holds an exclusive row-level lock for the duration of the transaction, serializing concurrent election attempts.

Decision Flow (runs every 10 seconds on every node)

```
Step 1 - Open transactional block with PESSIMISTIC - WRITELock Step 2 - Read the LEADER_O
```


C
K
ro
w
vi
a
S
EL
E
C
T
F
O
R
U
P
D
AT
E
(bl
oc
ks
ot
he
rn
od
es
)
%,
ep
3
-
N
or
ow
e
xis
ts
->
cr
ea
te
ro
w
wit
h
ou
rn
od
el
D
+
10
s
ex
pir
y -

```
>
be
co
m
el
ea
de
r
%,
ep
4
-
R
ow
e
xis
ts,
n
od
e_
id
==
s
elf
-
> r
en
ew
e
xp
iry
-
>
co
nti
nu
e
as
le
ad
er
%,
ep
5
-
R
ow
e
xis
ts,
n
od
e_
id
!=
se
lf,
```

```

ex
pir
y
in
fut
ur
e -
>
do
n
ot
hi
ng
-
>
no
tl
es
@Transactional(isolation = Isolation.SERIALIZABLE)
public boolean tryAcquireLeadership() {
    String nodeId = nodeService.getCurrentNodeId();
    if (nodeId == null) return false;

    Optional<LeaderElection> lockOpt =
        repo.findByLockNameForUpdate("LEADER_LOCK"); // SELECT FOR UPDATE

    if (lockOpt.isEmpty()) {
        // First ever election - create the lock row
        LeaderElection lock = new LeaderElection();
        lock.setNodeId(nodeId);
        lock.setExpiresAt(LocalDateTime.now().plusSeconds(10));
        repo.save(lock);
        currentlyLeader = true;
        return true;
    }

    LeaderElection lock = lockOpt.get();

    if (lock.getNodeId().equals(nodeId)) {
        // Self renew - extend our lease
        lock.setExpiresAt(LocalDateTime.now().plusSeconds(10));
        repo.save(lock);
        return true;
    }

    if (LocalDateTime.now().isAfter(lock.getExpiresAt())) {
        // Steal expired lease from dead node
        lock.setNodeId(nodeId);
        // Standard read - no lock
        lock.setExpiresAt(LocalDateTime.now().plusSeconds(10));
        Optional<LeaderElection> findByName = repo.findByLockName(String lockName);
        repo.save(lock);
        currentlyLeader = true;
        // Acquires exclusive row lock - used during election
        return true;
    }
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT l FROM LeaderElection l WHERE l.lockName = :lockName")
    Optional<LeaderElection> findByNameForUpdate(String lockName);
    return false; // another node holds an active lease
}
}

```

Why Pessimistic vs Optimistic Locking?

Repository: SELECT FOR UPDATE

@Lock(LockModeType.PESSIMISTIC_WRITE) issues SELECT FOR UPDATE - an exclusive row lock held for the transaction. Two nodes simultaneously calling this will serialize: node A locks, reads, updates, commits; node B then reads the updated row and backs off. Optimistic locking (@Version) would throw ObjectOptimisticLockingFailureException requiring retry loops. Pessimistic is simpler and correct since contention is expected and brief.

```

        node.setLeader(false); database.insert(node);
        node.setRegisteredAt(LocalDateTime.now());
        // Replicate the name to primary to avoid duplicate heartbeat since {}",
        nodeRepository.save(node);
        @Modifying @Transactional @Query("update Node set now = now() where nodeid = :nodeid")
        this.currentNodeid = nodeid;
        @Query("update Node set now = now() where nodeid = :nodeid")
        log.info("Node registered: {} ", nodeid);
        void updateHeartbeat(String nodeId, LocalDateTime ts);
    }
}

```

8. STEP-BY-STEP: HEARTBEAT AND FAULT TOLERANCE

Dead Node Detection (every 10 seconds)

Heartbeat Loop (every 5 seconds)

100 Dispatch - Leader Only

Any node with a heartbeat timestamp older than 15 seconds is considered failed. The 15s threshold (3x the 5s

```
}return<E06800<Component /5</Layouts;from schemas
```

```
} application/json:
```

```
} do NOT hand edit generated files
```

```
# Any backend API change breaks the frontend at compile time
```

```
Step 5 - Recharts Area Chart with Gradient
```

```
Step 6 - Real Time Data with the Functional API
```

```
Step 7 - Real Time Data with the Functional API
```



```
COPY --from=build . /app/target/* -i app.jar
USER postgres: { condition: "service_healthy" }
EXPOSE 8080
```

Essential Docker Commands

12. REST API REFERENCE

All endpoints are under the `/api` context path. A valid session cookie is required for all endpoints except POST `/api/auth/login` and GET `/api/actuator/health`.

Authentication Endpoints

Method	Endpoint	Description
POST	<code>/api/auth/login</code>	Login: body {username, password} returns session cookie
POST	<code>/api/auth/logout</code>	Invalidate current session
GET	<code>/api/auth/me</code>	Return current authenticated user details

Jobs CRUD Endpoints

Method	Endpoint	Description
GET	<code>/api/jobs</code>	List jobs (query: page, size, status)
POST	<code>/api/jobs</code>	Create new job (body: name, cronExpression, jobType, payload)
GET	<code>/api/jobs/:id</code>	Get single job by ID
PUT	<code>/api/jobs/:id</code>	Update job (name, cronExpression, description, payload)
DELETE	<code>/api/jobs/:id</code>	Delete job and cascade its execution history
POST	<code>/api/jobs/:id/pause</code>	Change job status to PAUSED
POST	<code>/api/jobs/:id/resume</code>	Change job status to ACTIVE
POST	<code>/api/jobs/:id/trigger</code>	Manually trigger job execution immediately

Executions, Nodes, and Stats

Method	Endpoint	Description
GET	<code>/api/executions</code>	List executions (query: page, jobId, status)
GET	<code>/api/executions/:id</code>	Get execution detail including output and error log
GET	<code>/api/nodes</code>	List all registered scheduler nodes with heartbeat status
GET	<code>/api/cluster/leader</code>	Get current leader node information
GET	<code>/api/stats</code>	Dashboard stats: job counts, success rate, charts data
GET	<code>/api/actuator/health</code>	Spring Actuator health (no auth required)

Example: Create a Job

13. KEY DESIGN DECISIONS AND PATTERNS

Why Database Locking and Not Zookeeper or Redis?

Most production distributed systems use Zookeeper, etcd, or Redis for leader election and coordination. Using PostgreSQL SELECT FOR UPDATE achieves the same atomicity guarantee with zero additional infrastructure. Since the system already has a mandatory PostgreSQL dependency, this avoids adding a separate service. This approach demonstrates deep RDBMS knowledge and is a valid architecture for systems where the DB is the single shared state store. Tradeoff: does not work if the DB itself fails - but neither does the job system since jobs are stored there.

Why Session-Based Auth and Not JWT?

JWT is stateless and scales horizontally without shared session storage. However, for an admin-only scheduler dashboard, session-based auth is simpler, more secure by default (no token in localStorage), and cleanly demonstrates Spring Security configuration. JWT adds complexity: token expiry, refresh flows, and revocation lists. At single-admin scale, none of that complexity provides benefit.

Why Contract-First API Design with OpenAPI and Orval?

Writing the OpenAPI 3.0 spec before any code enforces an explicit API contract. Orval then generates type-safe React Query hooks automatically. Backend changes break the frontend at TypeScript compile time - not at runtime in the browser. This makes the integration between services mechanically correct rather than relying on manual coordination. The downside is the extra codegen step, but it pays off immediately in larger teams.

volatile boolean currentlyLeader

The currentlyLeader flag is written by the scheduler thread (in tryAcquireLeadership) and read by the job dispatch thread (in runScheduler). Without the volatile keyword, the JVM may cache this boolean in a CPU register or cache line, making the write invisible to the reading thread. volatile guarantees that every read sees the most recent write. Using AtomicBoolean or synchronized would also work but is unnecessary since the flag has only one writer.

Repository Pattern and Testability

Spring Data JPA repositories are interfaces only. Spring generates all SQL at startup from method names and @Query annotations. Services depend on repository interfaces, not implementations. In unit tests, Mockito can substitute the interface with a mock in milliseconds. No actual database is needed for service-layer unit tests. This separation is the key architectural decision that makes the codebase testable at scale.

14. RUNNING THE APPLICATION

Prerequisites

[93](#),
av
a
17
+ (
O
pe
nJ
D
K
or
Gr
aa
IV
M)
-
ch
ec
k: j
av
a -
ve
rsi
on
[94](#),
av
en
3.
9+
-
ch
ec
k:
m
vn
-v
er
si
on
[95](#),
od
e.j
s
20
+
an
d
pn
p
m
9+
-
ch

```

ec
k:
pn
p
m
-v
%,
os
tgr
eS
Q
L
15
+r
un
ni
ng
lo
ca
lly,
O
R
us
e
do

# 1. Set required database environment variables
export PGHOST=localhost
# Build and start 2 scheduler nodes + PostgreSQL
export PGPORT=5432
export PGDATABASE=scheduler
export PGUSER=postgres
export PGPASSWORD=yourpassword
# Run nodes in a specific host: localhost:8080/api
# Node 2 for Backend / Localhost:8080/api/viceTest
# 2. Start the Spring Boot Backend
mvn spring-boot:run -f backend/pom.xml
# -> API ready at http://localhost:8080/api
# -> Health check: http://localhost:8080/api/actuator/health
# -> Within 15 seconds, node-2 detects the expired lease
# Test breakdown: leadership automatically
# 3. In a new terminal, install deps and start frontend
pnpm install
# LeaderElectionServiceTest - 7 unit tests (election scenarios)
# Verifies new leader; 5 integration tests (MockMvc + H2)
# -> Dashboard at http://localhost:3000
curl -b cookies.txt http://localhost:8081/api/cluster/leader
# Default login credentials: admin / admin123

```

URL	What it shows
http://localhost:3000	React admin dashboard (login first)
http://localhost:8080/api/actuator/health	Backend health status (no auth)
http://localhost:8080/api/auth/me	Current session user (JSON)
http://localhost:8080/api/jobs	All jobs paginated (requires auth cookie)
http://localhost:8080/api/stats	Dashboard stats JSON (requires auth cookie)
http://localhost:8080/api/nodes	All cluster nodes and heartbeat status
http://localhost:8080/api/cluster/leader	Current leader node info